Sealed-Typen & Pattern Matching

Carsten Gips (HSBI)

Was nervt Sie an diesem Code?

```java
interface Expr {}
record IntLiteral(int value) implements Expr {}
record Add(Expr left, Expr right) implements Expr {}
record Negate(Expr inner) implements Expr {}

int evalOld(Expr e) {
    if (e instanceof IntLiteral) {
        IntLiteral lit = (IntLiteral) e;
        return lit.value();
    } else if (e instanceof Add) {
        Add add = (Add) e;
        return evalOld(add.left()) + evalOld(add.right());
    } else if (e instanceof Negate) {
        Negate neg = (Negate) e;
        return -evalOld(neg.inner());
    } else {
        throw new IllegalArgumentException("Unknown expression: " + e);
    }
}
```

Pattern Matching für `instanceof`

```java
// vor Java 16:
if (obj instanceof String) {
    String s = (String) obj;
    if (s.length() > 5) {
        IO.println(s.toUpperCase());
    }
}
```

```java
// modern:
if (obj instanceof String s && s.length() > 5) {
    IO.println(s.toUpperCase());
}
```

Switch Expressions

```
switch (day) {
    case MONDAY:
    case FRIDAY:
        IO.println("Almost weekend");
        break;
    default:
        IO.println("Another day");
}
```

```
String message = switch (day) {
    case MONDAY, FRIDAY -> "Almost weekend";
    case SATURDAY, SUNDAY -> "Weekend!";
    default -> "Another day";
};
```

Type Patterns im `switch`

```
sealed interface Expr permits IntLiteral, Add, Negate {}

record IntLiteral(int value) implements Expr {}
record Add(Expr left, Expr right) implements Expr {}
record Negate(Expr inner) implements Expr {}
```

```
int eval(Expr e) {
    return switch (e) {
        case IntLiteral lit -> lit.value();
        case Add add -> eval(add.left()) + eval(add.right());
        case Negate neg -> -eval(neg.inner());
    };
}
```

Guarded Patterns im `switch`

```
int sign(Expr e) {
    return switch (e) {
        case Expr.IntLiteral lit when lit.value() > 0 -> 1;
        case Expr.IntLiteral lit when lit.value() < 0 -> -1;
        case Expr.IntLiteral lit -> 0;
        default -> throw new IllegalArgumentException("Not a literal: " + e);
    };
}
```

Record-Patterns / Dekonstruktion

```java
record Point(int x, int y) {}
```

```java
static int manhattan(Object o) {
    if (o instanceof Point) {
        Point p = (Point) o;
        return Math.abs(p.x()) + Math.abs(p.y());
    }
    throw new IllegalArgumentException("Not a point: " + o);
}
```

Record-Patterns / Dekonstruktion

```
record Point(int x, int y) {}
```

```
static int manhattan(Object o) {
    if (o instanceof Point) {
        Point p = (Point) o;
        return Math.abs(p.x()) + Math.abs(p.y());
    }
    throw new IllegalArgumentException("Not a point: " + o);
}
```

```
static int manhattan(Object o) {
    return switch (o) {
        case Point(int x, int y) -> Math.abs(x) + Math.abs(y);
        default -> throw new IllegalArgumentException("Not a point: " + o);
    };
}
```

Wrap-Up

Für Datenhierarchien (z.B. `Expr`, `Shape`, `Command`) nutzen Sie heute:

- `record` für Daten,
- `sealed` für eine abgeschlossene Variantenmenge, und
- Pattern Matching im modernen `switch` (mit Arrow-Syntax) für knappen, typsicheren und gut wartbaren Code im (teilweise) funktionalen Stil.

Last modified: *8adc0c3 2026-06-03 patternmatching: add new screencast*